JPA collection-valued association

학습목표

Collection-Valued Association 구현
eager loading/lazy loading 정책 이해

목차

1. 배경지식 2
    1) Collection-Valued Association 구현 2
    2) Eager Loading / Lazy Loading 3
    3) context path 5
2. jpa4 프로젝트 6
    1) 프로젝트 생성 6
    2) build.gradle 파일 수정 8
    3) application.properties 9
    4) Department 엔터티 클래스 10
    5) Student 엔터티 클래스 생성 10
    6) Department 엔터티 클래스 수정 11
    7) Department 리포지토리 12
    8) DepartmentController 컨트롤러 13
    9) common.css 14
    10) department/list1.html 15
    11) department/list2.html 16
3. 순환참조 문제 19
    1) 순환 참조 19
    1) @JsonIgnore 19
    2) lombok 21
    3) 요약 21
    4) StackOverflowError 에러 메시지 22
4. 연습문제 23

1. 배경지식

1) Collection-Valued Association 구현

학과(department) 한 개에 소속된 학생은 여러명이다.
Department 객체에 소속 학생들 목록에 해당하는 속성을 구현할 수 있다.
구현해야만 하는 것이 아니고, 필요한 경우에만 구현하면 됨.

```
@OneToMany(mappedBy="department")
List<Student> students;
```

이때 Student 엔터티 클래스에 @ManyToOne department 속성이 구현되어 있어야 한다.
Single-Valued Association을 먼저 구현하고 Collection-Valued Association을 구현해야 한다.

관계가 있는 두 엔터티 클래스 사이의 관계 속성을 구현할 때,
외래키가 있는 엔터티 클래스에 @ManyToOne 속성을 구현하는 것이 먼저이다. (single valued association)
그리고 필요하다면, 그 반대쪽 클래스에 @OneToMany 속성을 구현할 수도 있다. (collection valued association)

Single-Valued Association 구현은 거의 필수이지만,
Collection-Valued Association 구현은 선택이다.
Department 객체로부터, 소속 학생 목록을 구할 필요가 있을 때만 구현한다.

@OneToMany

```
@Entity
public class Department {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    int id;
    String name;

    @OneToMany(mappedBy="department")
    List<Student> students;
```

학과 한 개에 학생 여려명이 소속되어 있다.
Student 클래스에 @ManyToOne department 속성이 구현되어 있다.
학과에 소속된 학생 목록

```
@OneToMany
(mappedBy="department")
List<Student> students;
```

@ManyToOne

```
@Entity
public class Student {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    int id;
    String name;

    @ManyToOne
    @JoinColumn(name = "departmentId")
    Department department;
```

2) Eager Loading / Lazy Loading

Student 엔터티 객체에는 다음과 같은 속성이 있다. (single-valued association)
  Department department;

학생 레코드를 조회해서 Student 엔터티 객체가 생성될 때, 소속 학과 정보도 department 속성에 자동으로 채워진다.

Department 엔터티 객체에는 다음과 같은 속성이 있다. (collection-valued association)
  List<Student> students;

학과 레코드를 조회해서 Department 엔터티 객체가 생성될 때, 소속 학생 목록도 students 속성에 자동으로 채워진다.

위와 같이 assocation 속성이 자동으로 채워지는 시점이 언제인가를 기준으로
eager loading 정책과 lazy loading 정책이 있다.

eager loading

eager loading 정책은 DB에서 학생 데이터를 조회해서 Student 엔터티 객체를 생성할 때,
department 속성에 채울 소속 학과 정보도 DB에서 같이 조회해서
department 속성에 채운다.

single-valued association은 eager loading 정책일 확률이 높다.

어떤 엔터티 객체 데이터를 DB에서 조회할 때,
그 엔터티의 모든 association 속성을 다 조회한다. (예: Student 엔터티의 department 속성)

lazy loading

lazy loading 정책은 DB에서 학과 데이터를 조회해서 Department 엔터티 객체를 생성할 때,
students 속성에 채울 소속 학생 목록은 DB에서 같이 조회하지 않는다.

나중에 students 속성값을 리턴하는 getter 메소드(getStudents() 메소드)가 호출되면
소속 학생 목록을 DB에서 조회해서 students 속성에 채우고 리턴한다.
만약 students 속성의 getter 메소드가 호출되지 않으면,
소속 학생 목록은 DB에서 조회되지 않는다.

collection-valued association은 lazy loading 정책일 확률이 높다.

lazy loading 정책은, 어떤 엔터티 객체 데이터를 DB에서 조회할 때, (예: Department 엔터티)
그 엔터티의 association 속성을 미리 조회하지 않고, 뒤로 미룬다. (예: Department 엔터티의 students 속성)

Eager Loading 정책의 장점
어차피 시장에 왔으니, 필요할 것 같은 상품들을 다 사오는 것이 편하다.
어차피 DB에 연결해서 명령을 실행하고 데이터를 받아오고 있으니, 필요할 것 같은 데이터들은 한 번에 다 가져오는 것이 좋다.

Eager Loading 정책의 단점
사온 상품이 필요 없다는 것을 알게 되면 돈 낭비다.
가져온 데이터를 결국 사용하지 않게 된다면, 자원 낭비다.

Lazy Loading 정책의 장점
비록 지금 시장에 왔지만, 현재 필요하다는 확신이 없는 상품을 사지 않는다.
학과 레코드를 조회해서 Department 엔터티 객체에 채울 때,
그 학과 소속 학생 목록 레코드들을 조회하지 않는다.
돈 낭비, 자원 낭비를 줄인다.

Lazy Loading 정책의 단점
아까 시장에서, 확신이 없어서 사지 않았던 상품이, 지금 필요하면
다시 시장에 가서 사와야 한다. 시간이 걸린다.
그 학과 소속 학생 목록이 필요하면,
또 DB에 연결해서 그 학과 소속 학생 레코드들을 조회해야 한다.
시간이 걸린다.

정책 설정
JPA 엔진에 따라 다르지만,
Single-Valued Association 객체 생성은 Eager Loading 인 경우가 많고
Collection-Valued Association 객체 생성은 Lazy Loading 인 경우가 많다.

이 정책을 선택하는 것은 성능 상의 차이만 있을뿐 기능상의 차이는 없다.
Lazy Loading / Eager Loading 정책의 차이는 언제 데이터 조회를 하는가 이다.

Lazy Loading 정책은, 데이터 조회를 가급적 미루자는 것일 뿐이다.
두 정책 모두, 데이터를 사용할 때는 언제나 그 데이터가 이미 조회되어 있다.
따라서 기능상의 차이는 없다.

```
@ManytoOne(fetch=FetchType.LAZY)
@ManytoOne(fetch=FetchType.EAGER)

@OneToMany(mappedBy="department", fetch=FetchType.LAZY)
@OneToMany(mappedBy="department", fetch=FetchType.EAGER)
```

위와 같이 FetchType.LAZY, FetchType.EAGER 를 설정해 줄 수 있다.
그렇지만, 이 설정이 언제나 지켜진다는 보장은 없다.
JPA 엔진을 구현하는 사람 맘대로이다.

3) context path

URL의 구성 요소는 다음과 같다.

프로토콜://서버주소:포트번호/contextPath/디렉토리경로/파일이름?queryString

예) http://localhost:8088/student/list?pg=1&sz=15 context path: /
http://localhost:8088/jpa1/student/list?pg=1&sz=15 context path: /jpa1

절대 경로 URL

이렇게 URL 전체를 다 적은 것은 절대 경로 URL 이라고 한다.
그런데 코딩할 때, 서버 주소 부분은 생략하고 다음과 같이 URL을 적는다.

예) /student/list?pg=1&sz=15

이렇게 서버 주소 부분만 생략해서, / 문자로 시작하는 URL도 절대 경로 URL 이다.

```
<link rel="stylesheet" type="text/css" href="/common.css" />
```

URL이 / 문자로 시작하는 경우,
이것은 서버 주소 부분만 생략한 절대 경로 URL이고
URL의 첫 문자인 / 부분이 context path 이다.

spring boot 프로젝트에서 context path는 기본으로 / 이다.
그런데 서버에 프로젝트를 설치하는 방법에 따라,
context path 부분이 / 일수도 있고, /프로젝트이름 으로 바뀔 수도 있다.

```
<link rel="stylesheet" type="text/css" href="/jpa4/common.css" />
```

만약 플젝을 서버에 설치하면서 context path 부분이 /jpa4으로 변경되면
소스코드의 URL 부분을 위와 같이 수정해야 한다.
이렇게 수정해야 하는 것이 조금 귀찮다.

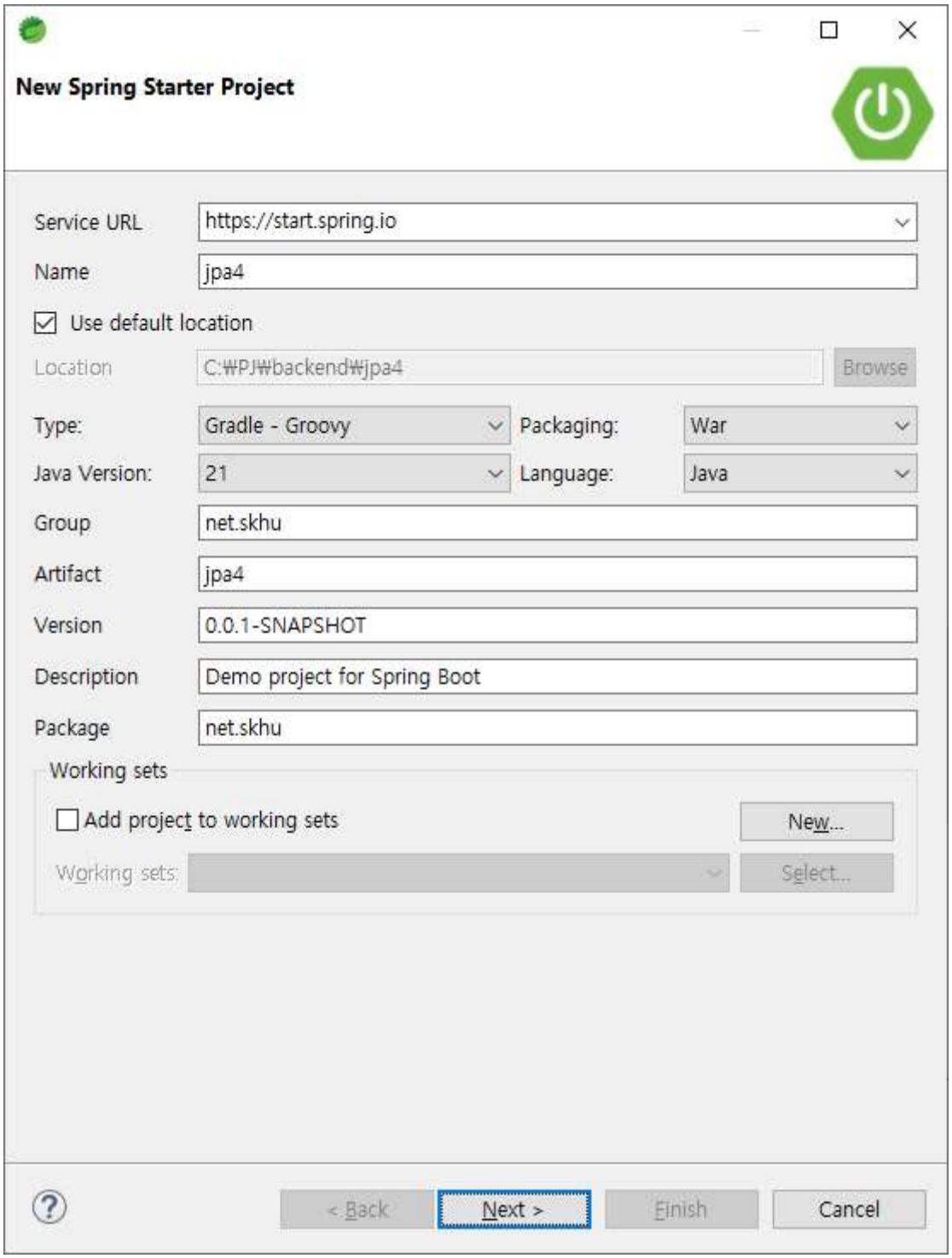
```
<link rel="stylesheet" type="text/css" th:href="@{/common.css}" />
```

위와 같이 구현하면 thymeleaf가 context path 부분을 자동으로 수정해 줘서 편하다.
절대 경로 URL을 입력할 때, th:href="@{/common.css}" 형태로 입력하면,
(앞부분에 th: 을 붙이고, URL을 @{ } 괄호로 묶는다)
thymleaf가 context path 부분을 자동으로 수정해 준다.

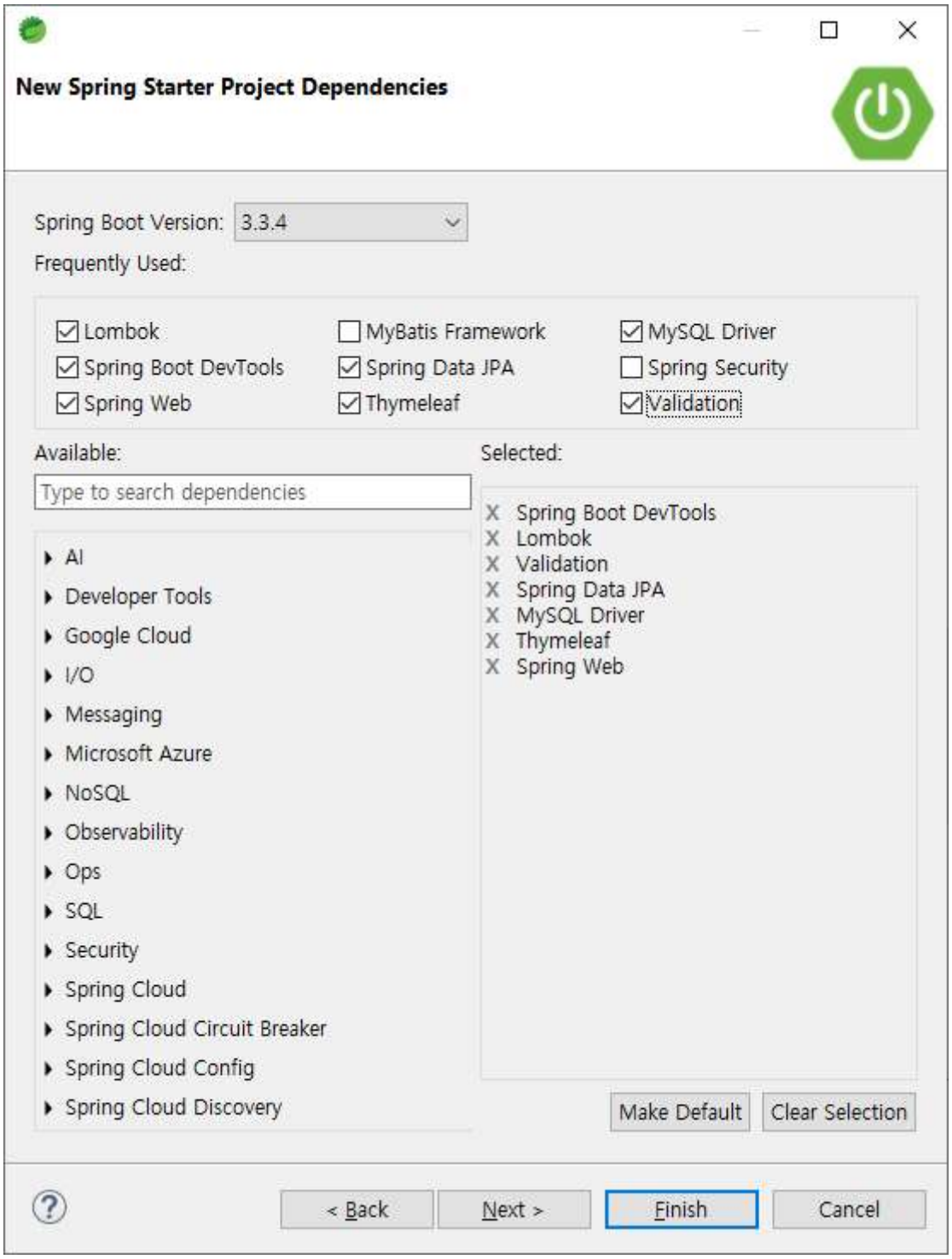
context path 부분이 없이 알파벳으로 시작하는 상대경로 URL은
위와 같이 구현할 필요 없고,
그냥 href="common.css" 형태로 구현하면 된다.

2.jpa4 프로젝트

1) 프로젝트 생성



Name	jpa4
Type	Gradle - Groovy
Package	net.skhu



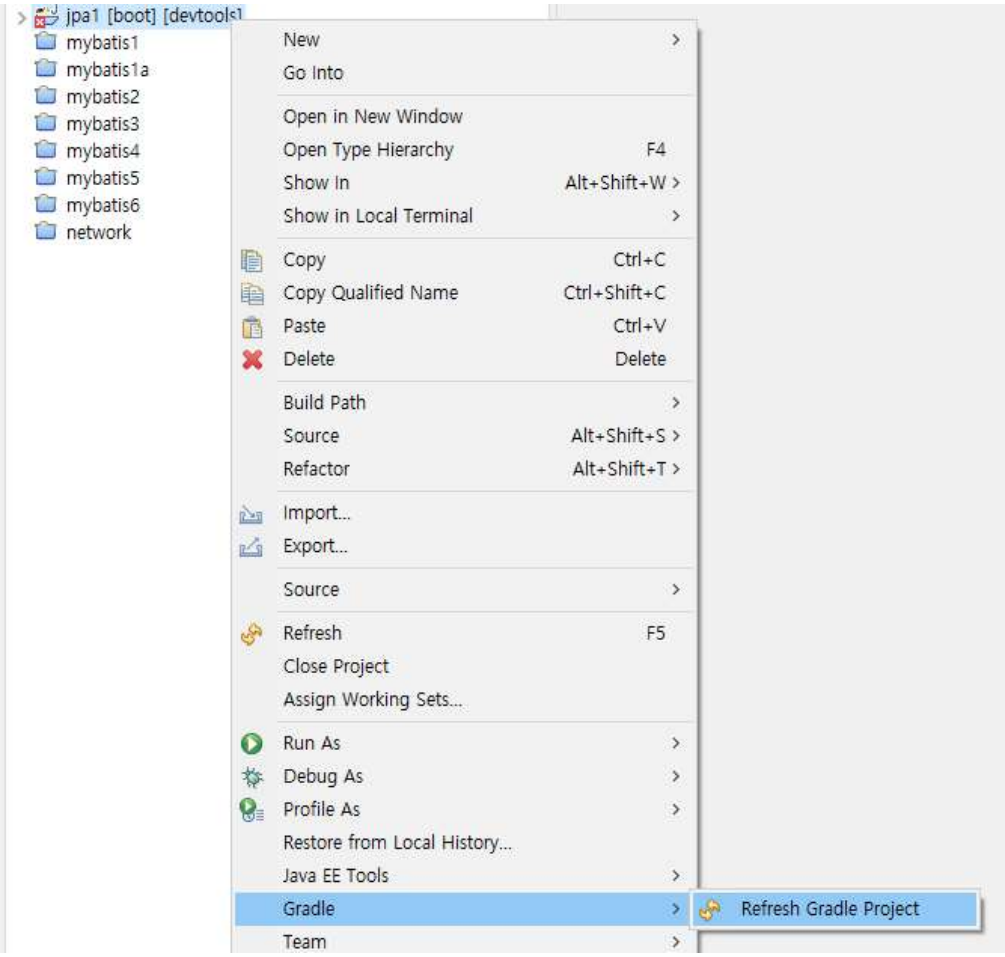

```
Spring Boot DevTools
Lombok
Validation
Spring Data JPA
MySQL Driver
Thymeleaf
Spring Web
```

2) build.gradle 파일 수정

```
dependencies {
  ... 생략 ...
    implementation 'org.modelmapper:modelmapper:3.2.5'
}
```

gradle 프로젝트의 설정 파일인 build.gradle 파일에
  modelmapper maven dependency 한 줄을 추가

Referesh Gradle Project



build.gradle 파일을 수정한 후에는 반드시 Refresh Gradle Project 해야 한다.

3) application.properties

```
spring.application.name=jpa4
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/student2?useUnicode=yes&characterEncoding=UTF-8&allowMultiQueries=true&serverTimezone=Asia/Seoul
spring.datasource.username=user1
spring.datasource.password=skhuA+4.5
spring.jpa.hibernate.naming.physical-strategy=org.hibernate.boot.model.naming.PhysicalNamingStrategyStandardImpl
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
logging.level.org.hibernate.orm.jdbc.bind=TRACE
logging.level.net.skhu=DEBUG
server.port=8088
```

```
spring.jpa.show-sql=true
```
JPA가 자동으로 생성하여 실행되는 SQL을 로그(log)에 출력하라는 설정.
STS에서 실행할 때, 로그는 console 창에 출력된다.

```
spring.jpa.properties.hibernate.format_sql=true
```
로그(log)에 출력되는 SQL을 보기 좋게 포맷하라는 설정.

이 설정을 끄려면, 줄 선두에 # 문자를 넣으면 된다.
문자부터 줄 끝까지는 주석(comment)이 된다.

4) Department 엔터티 클래스

src/main/java/net/skhu/entity/Department.java

```java
package net.skhu.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import lombok.Data;

@Data
@Entity
public class Department {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    int id;

    String name;
    String shortName;
    String phone;
}
```

5) Student 엔터티 클래스 생성

src/main/java/net/skhu/entity/Student.java

```java
package net.skhu.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import lombok.Data;

@Data
@Entity
public class Student {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    int id;

    String studentNo;
    String name;
    String phone;
    String sex;
    String email;

    @ManyToOne
    @JoinColumn(name="departmentId")
    Department department;
}
```

single-valued association을 먼저 구현해야 한다.

6) Department 엔터티 클래스 수정

src/main/java/net/skhu/entity/Department.java

```java
package net.skhu.entity;

import java.util.List;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.OneToMany;
import lombok.Data;

@Data
@Entity
public class Department {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    int id;

    String name;
    String shortName;
    String phone;

    @OneToMany(mappedBy="department")
    List<Student> students;
}
```

single-valued association을 먼저 구현한 후,
collection-valued association을 구현해야 한다.

7) Department 리포지토리

src/main/java/net/skhu/repository/DepartmentRepository.java

```
1  package net.skhu.repository;
2
3  import org.springframework.data.jpa.repository.JpaRepository;
4
5  import net.skhu.entity.Department;
6
7  public interface DepartmentRepository extends JpaRepository<Department, Integer>  {
8
9  }
```

8) DepartmentController 컨트롤러

src/main/java/net/skhu/controller/DepartmentController.java

```java
package net.skhu.controller;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import net.skhu.repository.DepartmentRepository;

@Controller
public class DepartmentController {

    @Autowired DepartmentRepository departmentRepository;

    @GetMapping("department/list1")
    public String list1(Model model) {
        model.addAttribute("departments", departmentRepository.findAll());
        return "department/list1";
    }

    @GetMapping("department/list2")
    public String list2(Model model) {
        model.addAttribute("departments", departmentRepository.findAll());
        return "department/list2";
    }
}
```

list1 액션 메소드 구현과 list2 액션 메소드 구현은
리턴하는 뷰의 이름만 다르고 구현이 같다.

9) common.css

src/main/resources/static/common.css

```
div.container { width: 800px; margin: 10px auto; font-size: 11pt; }
.shadow { box-shadow: 2px 2px 5px gray; }

.btn { padding: 0.4em 1em; border: 1px solid gray;
       border-radius: 0.5em; background: linear-gradient(#fff, #ddd);
       text-decoration: none; color: black;
       display: inline-block; }
.btn:active {
      -ms-transform: translateY(2px);
      -webkit-transform: translateY(2px);
      transform: translateY(2px);
      background: #ccc; }

table.list { border-collapse: collapse; width: 100%; }
table.list td { padding: 4px; border: 1px solid gray; }
table.list th { padding: 4px; border: 1px solid gray;  background-color: #eee; }

div.row { margin-bottom: 0.8em; }
div.row > label:nth-child(1) { display: inline-block; min-width: 4em; text-align: right; }

input[type=text], input[type=number] { padding: 0.3em; }
select { padding: 0.3em; }
h1 { margin: 0.5em 0; }
hr { margin: 1em 0; }
.error { color: red; padding: 0.2em; margin-left: 1em; }

div.pagination table { border-collapse: collapse; margin-top: 1em; }
div.pagination table td { border: 1px solid #ccc; text-align: center; }
div.pagination table td.active { background-color: #48f; }
div.pagination table a { text-decoration: none; color: blue; display: inline-block;
                         width: 25px; padding: 6px; }
div.pagination table td.active a { color: white; }
```

10) department/list1.html

src/main/resources/templates/department/list1.html

```
<!doctype html>
<html lang="ko" xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="utf-8">
  <link rel="stylesheet" type="text/css" th:href="@{/common.css}" />
  <style>
  </style>
</head>
<body>
<div class="container">
  <h1>학과목록</h1>
<table class="list">
  <thead>
    <tr>
    <th>id</th>
    <th>학과명</th>
    <th>단축명</th>
    <th>전화</th>
    </tr>
  </thead>
  <tbody>
    <tr th:each="dp : ${ departments }">
      <td th:text="${ dp.id }"></td>
      <td th:text="${ dp.name }"></td>
      <td th:text="${ dp.shortName }"></td>
      <td th:text="${ dp.phone }"></td>
    </tr>
  </tbody>
  </table>
</div>
</body>
</html>
```

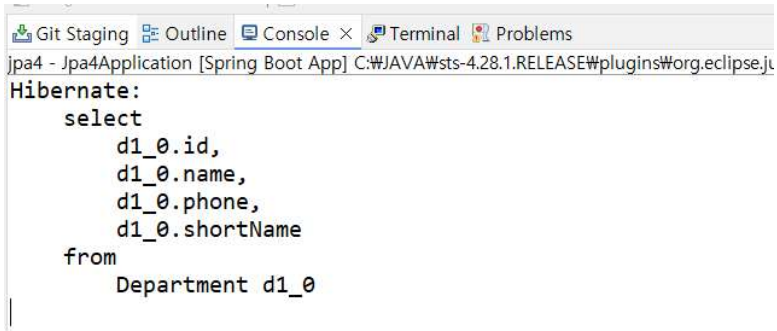
departments 는 액션 메소드가 전달한 model attribute 이름이다.
departments 학과 목록의 학과 각각을 dp 변수에 대입하면서
tr 태그와 그 아래 td 태그들이 반복 출력된다.

id, name, shortName, phone 부분은 학과(Department) 객체의 속성이다.

학과목록

id	학과명	단축명	전화
1	소프트웨어공학과	소프	02-2610-9563
2	컴퓨터공학과	컴공	02-2610-9565
3	정보통신공학과	정통	02-2610-9568
4	글로컬IT공학과	글티	02-2610-9569

http://localhost:8088/department/list1



학과 목록 조회 SQL만 한 번 실행되고, 학생 목록 조회 SQL 명령은 실행되지 않는다.
소속 학생 목록 colection-valued association은 lazy loading 이기 때문이다.

11) department/list2.html

src/main/resources/templates/department/list2.html

```
<!doctype html>
<html lang="ko" xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="utf-8">
  <link rel="stylesheet" type="text/css" th:href="@{/common.css}" />
  <style>
    h3 { margin-bottom: 2px; }
  </style>
</head>
<body>
<div class="container">
  <h1>학과별 학생목록</h1>
  <div th:each="dp : ${ departments }">
    <h3 th:text="${ dp.name }"></h3>
    <table class="list">
    <thead>
      <tr>
      <th>id</th>
      <th>학번</th>
      <th>이름</th>
      <th>학과</th>
      <th>전화</th>
      <th>성별</th>
      <th>이메일</th>
      </tr>
    </thead>
    <tbody>
      <tr th:each="st : ${ dp.students }">
        <td th:text="${ st.id }"></td>
        <td th:text="${ st.studentNo }"></td>
        <td th:text="${ st.name }"></td>
        <td th:text="${ st.department.name }"></td>
        <td th:text="${ st.phone }"></td>
        <td th:text="${ st.sex }"></td>
        <td th:text="${ st.email }"></td>
      </tr>
    </tbody>
    </table>
  </div>
</div>
</body>
</html>
```

마치 java의 중첩된 for 반복문 처럼,
th:each="변수 : ${ 목록}" 반복 지시자가 중첩되었다.

```
<div th:each="dp : ${ departments }">      departments 학과 목록의 학과 각각에 대해서 반복
   <tr th:each="st : ${ dp.students }">    dp.students 학생 목록의 학생 각각에 대해서 반복
    <td th:text="${ st.id }"></td>
    <td th:text="${ st.studentNo }"></td>
    <td th:text="${ st.name }"></td>
    <td th:text="${ st.department.name }"></td>
```

위 코드의 구조는 아래 Java 코드의 구조와 유사하다

```
for (Department dp : departments)
    for (Student st : dp.getStudents()) {
        System.out.println(st.getId());
        System.out.println(st.getStudentNo());
        System.out.println(st.getName());
        System.out.println(st.getDepartment().getName());
    }
```

http://localhost:8088/department/list2

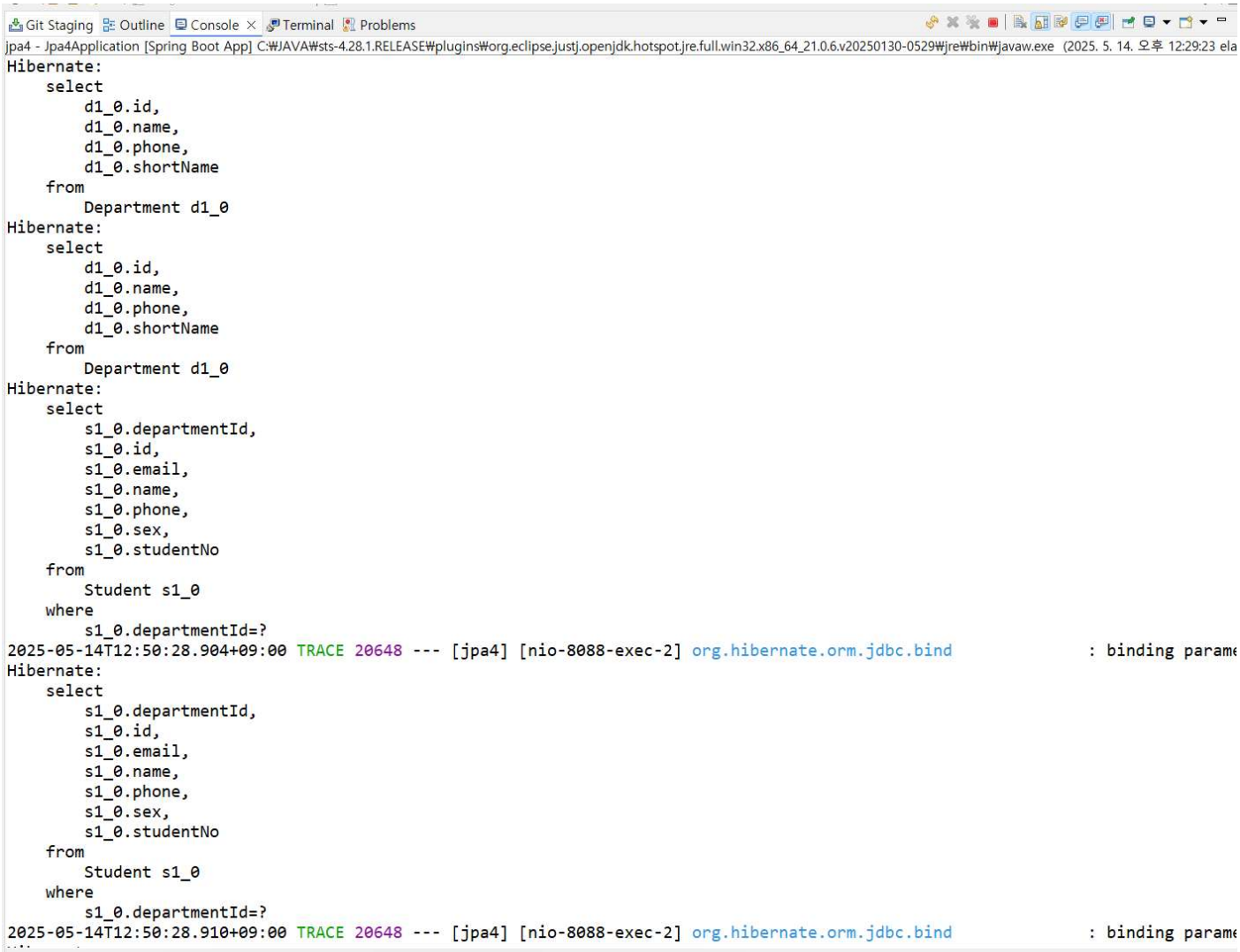
학과별 학생목록

소프트웨어공학과

id	학번	이름	학과	전화	성별	이메일
39	201632014	황찬금	소프트웨어공학과	010-5535-3424	남	hwang145@skhu.ac.kr
47	201632019	이하정	소프트웨어공학과	010-8598-3839	여	lee198@skhu.ac.kr
50	201634020	이복성	소프트웨어공학과	010-8473-3925	남	lee208@skhu.ac.kr
52	201632022	박남기	소프트웨어공학과	010-9229-4003	남	park229@skhu.ac.kr
60	201632030	전세승	소프트웨어공학과	010-3727-4761	남	cheon303@skhu.ac.kr
62	201732007	김택용	소프트웨어공학과	010-5911-4844	남	kim075@skhu.ac.kr
65	201732010	이영시	소프트웨어공학과	010-5366-4880	여	lee105@skhu.ac.kr
67	201732012	신춘동	소프트웨어공학과	010-3227-4969	남	sin123@skhu.ac.kr
69	201732014	윤주동	소프트웨어공학과	010-4636-5000	남	yoon144@skhu.ac.kr
70	201732015	김환정	소프트웨어공학과	010-4183-5008	남	kim154@skhu.ac.kr
79	201732020	장언서	소프트웨어공학과	010-9410-5478	남	jang209@skhu.ac.kr
86	201732026	송원기	소프트웨어공학과	010-1176-5806	남	song261@skhu.ac.kr
91	201832006	신순재	소프트웨어공학과	010-1042-6235	남	sin061@skhu.ac.kr
92	201832007	심영훈	소프트웨어공학과	010-5635-6268	남	sim075@skhu.ac.kr
94	201832009	정리두	소프트웨어공학과	010-4991-6330	남	jeong094@skhu.ac.kr
98	201832013	기도형	소프트웨어공학과	010-5719-6630	남	ki135@skhu.ac.kr
104	201832017	여희경	소프트웨어공학과	010-3913-6827	여	yeo173@skhu.ac.kr
120	201832030	김승영	소프트웨어공학과	010-8682-7816	남	kim308@skhu.ac.kr
124	201932009	조화병	소프트웨어공학과	010-2497-8019	여	ko092@skhu.ac.kr
129	201932014	이상은	소프트웨어공학과	010-7659-8206	남	lee147@skhu.ac.kr
133	201932016	도재인	소프트웨어공학과	010-7164-8413	남	do167@skhu.ac.kr
135	201932017	김준호	소프트웨어공학과	010-4717-8565	남	kim174@skhu.ac.kr
139	201932020	함헌석	소프트웨어공학과	010-8970-8993	남	ham208@skhu.ac.kr
142	201932022	박준홍	소프트웨어공학과	010-2673-9155	남	park222@skhu.ac.kr
144	201932024	박환인	소프트웨어공학과	010-2515-9284	남	park249@skhu.ac.kr
146	201932026	유옥병	소프트웨어공학과	010-3615-9437	남	yu263@skhu.ac.kr
147	201932027	고나진	소프트웨어공학과	010-3603-9475	남	ko273@skhu.ac.kr
149	201932029	문병수	소프트웨어공학과	010-7894-9541	남	moon297@skhu.ac.kr
150	201932030	박래용	소프트웨어공학과	010-5871-9549	남	park305@skhu.ac.kr
157	202032012	김주영	소프트웨어공학과	010-4881-9651	남	kim124@skhu.ac.kr
175	123123123	김김	소프트웨어공학과	010-000-0000	남	a@a.a

컴퓨터공학과

id	학번	이름	학과	전화	성별	이메일
17	201532019	최현욱	컴퓨터공학과	010-7312-2171	남	choi197@skhu.ac.kr
18	201432019	임봉영	컴퓨터공학과	010-7475-2192	남	lim197@skhu.ac.kr
19	201532020	장영만	컴퓨터공학과	010-1527-2270	남	jang201@skhu.ac.kr
21	201532021	황하지	컴퓨터공학과	010-6682-2463	남	hwang216@skhu.ac.kr
26	201532026	김철지	컴퓨터공학과	010-1611-2712	남	kim261@skhu.ac.kr
32	201632007	박남영	컴퓨터공학과	010-8388-2997	남	park078@skhu.ac.kr
33	201632008	이식동	컴퓨터공학과	010-7392-3031	남	lee087@skhu.ac.kr



이클립스 콘솔창에서 실행되는 SQL 명령을 볼 수 있다.

학과 목록을 조회하는 SQL 명령이 1번 실행되었고,
학과 id 1, 2, 3, 4 소속 학생 목록을 조회하는 SQL 명령이 4번 실행된 것을 볼 수 있다.

학과 레코드가 4 개이므로, 학생 목록 조회도 4번 실행되었다. (lazy loading)

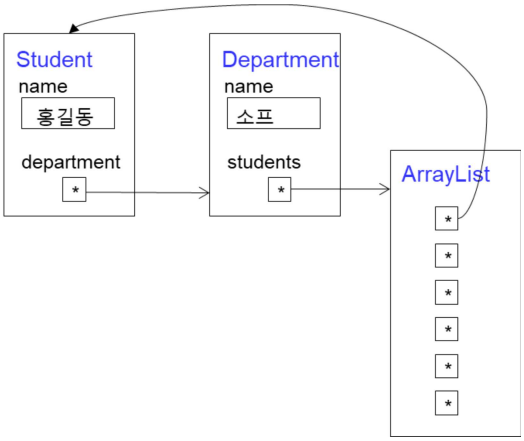
3. 순환참조 문제

1) 순환 참조

collection-valued association을 구현하면 순환 참조가 발생한다.

예를 들어 앞에서 설명한 student - department 관계 속성에서,
Student 객체의 department 속성값은 Department 객체 한 개이고, (single-valued association)
Department 객체의 students 속성값은 Student 객체 목록이다. (collection-valued association)

예를 들어,
'홍길동' 학생 객체의 department 속성값은, '소프트웨어공학과' 객체를 가르키는 참조이다.
'소프트웨어공학과' 객체의 stduents 속성값은, 이 학과 소속 학생에 대한 참조 목록이고,
이 참조들 중 하나는 '홍길동' 객체를 가르킨다.
즉 순환 참조이다.



이 객체들을 문자열로 표현할 때, 이 순환 참조 때문에 무한 재귀호출이 발생한다.
무한 재귀호출은 StackOverflowError 에러를 일으킨다.

무한 재귀의 예

위에서 예를 들은 '홍길동' 학생 객체에 대한 JSON 형태의 문자열을 만들어보자.

```
{ id: 1, name: "홍길동", department: { ... } }
```

... 으로 생략한 부분에 '소프트웨어공학과' 객체 값이 들어가야 한다.

```
{ id: 1, name: "홍길동", department: { id: 1, name: "소프트웨어공학과", students: [ ... ] } }
```

... 으로 생략한 부분에, 이 학과 소속 학생들 목록이 들어가야 한다.
이 학생 목록의 선두에 홍길동 학생이 있다면

```
{ id: 1, name: "홍길동", department: { id: 3, name: "소프트웨어공학과", students: [ { id: 1, name: "홍길동", department: {...}, ... ] } }
```

... 으로 생략한 부분에 '홍길동' 학생의 소속 학과인 '소프트웨어공학과' 객체 값이 또 들어가야 한다.
그 Department 객체 값에는 또 소속 학생들 객체 목록이 또 들어가야 한다.

이렇게 순환 참조를 문자열로 표현할 때, 꼬리에 꼬리를 무는 무한 반복이 발생하게 된다.
이 무한 반복 때문에 무한 재귀 호출이 발생하고,
무한 재귀호출은 StackOverflowError 에러를 일으킨다.

1) @JsonIgnore

위와 같은 StackOverflowError를 막으려면,
순환 참조 구조를 문자열로 표현할 때, @OneToMany 속성을 무시해야 한다.
위 구조에서 Department 객체의 students 속성이 @OneToMany 이다.
이 속성에 @JsonIgore 어노테이션을 붙이면,
Department 객체를 JSON 문자열로 변환할 때, 이 속성이 무시된다.

```
{ id: 1, name: "홍길동", department: { ... } }
```
... 으로 생략한 부분에 '소프트웨어공학과' 객체 값이 들어가야 한다.

```
{ id: 1, name: "홍길동", department: { id: 1, name: "소프트웨어공학과" } }
```
students 속성을 무시했더니, 순환 구조가 발생하지 않고, JSON 문자열 만들기가 끝났다.

2) lombok

@ToString.Exclude

@Data 어노테이션을 붙이면, lombok에 의해 toString 메소드가 자동으로 구현된다.
이 toString 메소드는 객체의 모든 속성값을 문자열로 만들어서 리턴한다.
이때도 @OneToMany 속성이 무시되지 않으면, 무한 재귀호출 문제가 발생한다.

@OneToMany 속성 앞에 @ToString.Exclude 어노테이션을 붙이면,
lombok이 toString 메소드를 자동으로 구현해 줄 때, 이 속성을 무시한다.

@EqualsAndHashCode.Exclude

lombok에 의해 equals, hashCode 메소드도 자동으로 구현된다.
여기서도 collection valued associated은 제외되어야 한다.

3) 요약

RestController를 구현한다면,
@OneToMany 속성에 @JsonIgnore를 붙여야 한다.

엔터티 객체의 toString 메소드를 호출할 거라면,
@OneToMany 속성에 @ToString.Exclude, @EqualsAndHashCode.Exclude를 붙여야 한다.

그런데, 디버깅을 위한 로그 기록에서,
엔터티 객체의 toString 메소드가, 나도 모르게 호출될 수 있다.

@OneToMany 속성에
@JsonIgore과 @ToString.Exclude를
언제나 붙이자.

```java
package net.skhu.entity;

import java.util.List;

import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.OneToMany;

import com.fasterxml.jackson.annotation.JsonIgnore;

import lombok.Data;
import lombok.ToString;

@Data
@Entity
public class Department {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    int id;

    String name;
    String shortName;
    String phone;

    @JsonIgnore
    @ToString.Exclude
    @EqualsAndHashCode.Exclude
    @OneToMany(mappedBy="department")
    List<Student> students;
}
```

4) StackOverflowError 에러 메시지

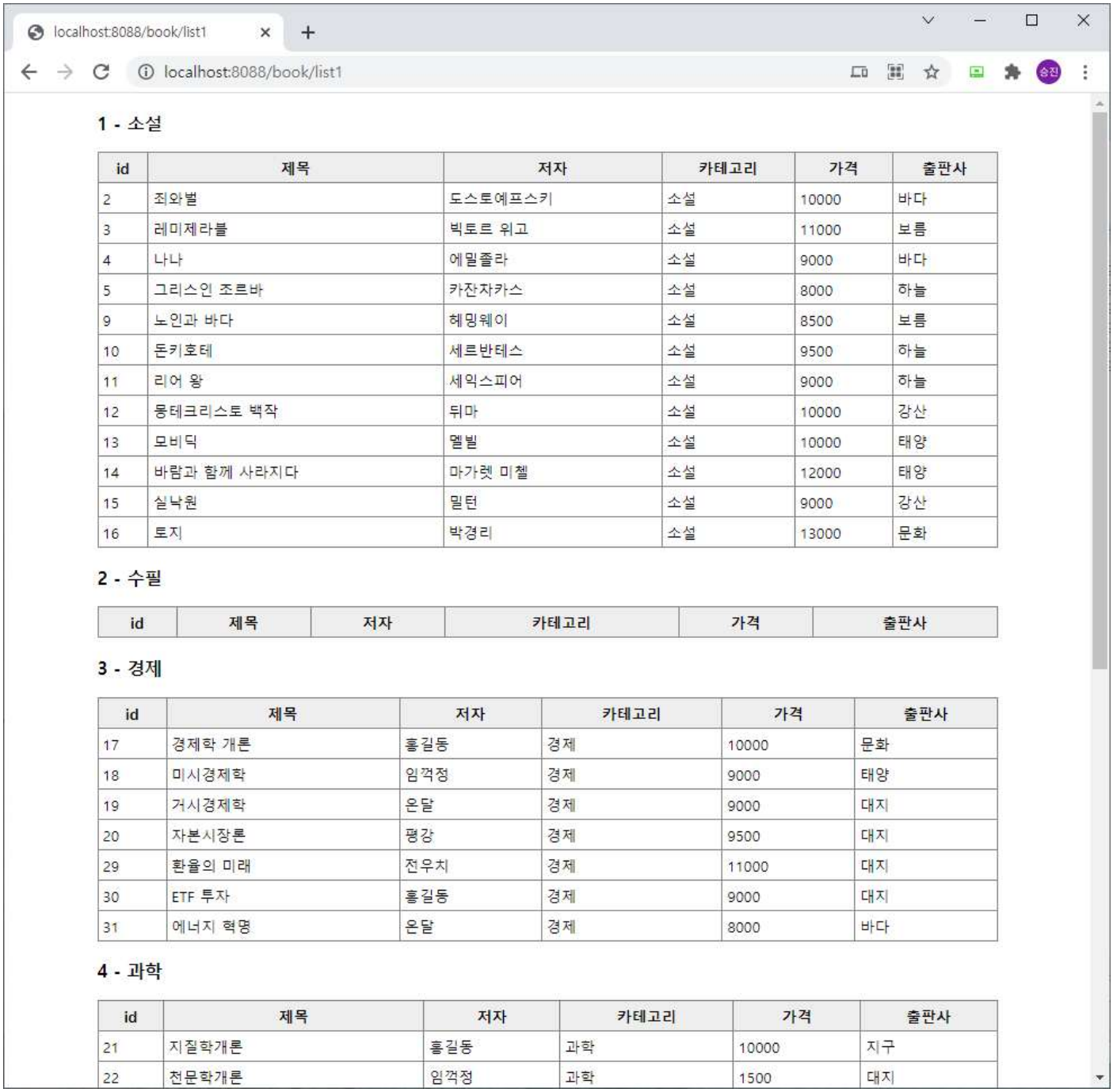
```
java.lang.StackOverflowError: null
        at java.base/java.lang.AbstractStringBuilder.append(AbstractStringBuilder.java:582) ~[na:n
        at java.base/java.lang.StringBuilder.append(StringBuilder.java:175) ~[na:na]
        at java.base/java.lang.StringBuilder.append(StringBuilder.java:87) ~[na:na]
        at java.base/java.lang.AbstractStringBuilder.<init>(AbstractStringBuilder.java:112) ~[na:n
        at java.base/java.lang.StringBuilder.<init>(StringBuilder.java:127) ~[na:na]
        at net.skhu.entity.Student.toString(Student.java:12) ~[classes/:na]
        at java.base/java.lang.String.valueOf(String.java:3365) ~[na:na]
        at java.base/java.lang.StringBuilder.append(StringBuilder.java:169) ~[na:na]
        at java.base/java.util.AbstractCollection.toString(AbstractCollection.java:457) ~[na:na]
        at org.hibernate.collection.internal.PersistentBag.toString(PersistentBag.java:622) ~[hibe
        at java.base/java.lang.String.valueOf(String.java:3365) ~[na:na]
        at java.base/java.lang.StringBuilder.append(StringBuilder.java:169) ~[na:na]
        at net.skhu.entity.Department.toString(Department.java:13) ~[classes/:na]
        at java.base/java.lang.String.valueOf(String.java:3365) ~[na:na]
        at java.base/java.lang.StringBuilder.append(StringBuilder.java:169) ~[na:na]
        at net.skhu.entity.Student.toString(Student.java:12) ~[classes/:na]
        at java.base/java.lang.String.valueOf(String.java:3365) ~[na:na]
        at java.base/java.lang.StringBuilder.append(StringBuilder.java:169) ~[na:na]
        at java.base/java.util.AbstractCollection.toString(AbstractCollection.java:457) ~[na:na]
        at org.hibernate.collection.internal.PersistentBag.toString(PersistentBag.java:622) ~[hibe
        at java.base/java.lang.String.valueOf(String.java:3365) ~[na:na]
        at java.base/java.lang.StringBuilder.append(StringBuilder.java:169) ~[na:na]
        at net.skhu.entity.Department.toString(Department.java:13) ~[classes/:na]
        at java.base/java.lang.String.valueOf(String.java:3365) ~[na:na]
        at java.base/java.lang.StringBuilder.append(StringBuilder.java:169) ~[na:na]
        at net.skhu.entity.Student.toString(Student.java:12) ~[classes/:na]
        at java.base/java.lang.String.valueOf(String.java:3365) ~[na:na]
        at java.base/java.lang.StringBuilder.append(StringBuilder.java:169) ~[na:na]
        at java.base/java.util.AbstractCollection.toString(AbstractCollection.java:457) ~[na:na]
        at org.hibernate.collection.internal.PersistentBag.toString(PersistentBag.java:622) ~[hibe
        at java.base/java.lang.String.valueOf(String.java:3365) ~[na:na]
        at java.base/java.lang.StringBuilder.append(StringBuilder.java:169) ~[na:na]
        at net.skhu.entity.Department.toString(Department.java:13) ~[classes/:na]
        at java.base/java.lang.String.valueOf(String.java:3365) ~[na:na]
        at java.base/java.lang.StringBuilder.append(StringBuilder.java:169) ~[na:na]
        at net.skhu.entity.Student.toString(Student.java:12) ~[classes/:na]
        at java.base/java.lang.String.valueOf(String.java:3365) ~[na:na]
```

위 에러 메시지를 잘 살펴보면,
Student, Department 클래스의 toString 메소드 무한 재귀 호출을 볼 수 있다.

2. 연습문제

department/list1 페이지와 동일한 구조로 book/list 페이지를 구현하라.
지난 과제 프로젝트에 추가로 구현하라.

http://localhost:8088/book/list1



1 - 소설

id	제목	저자	카테고리	가격	출판사
2	죄와벌	도스토예프스키	소설	10000	바다
3	레미제라블	빅토르 위고	소설	11000	보름
4	나나	에밀졸라	소설	9000	바다
5	그리스인 조르바	카잔차카스	소설	8000	하늘
9	노인과 바다	헤밍웨이	소설	8500	보름
10	돈키호테	세르반테스	소설	9500	하늘
11	리어 왕	셰익스피어	소설	9000	하늘
12	몽테크리스토 백작	뒤마	소설	10000	강산
13	모비딕	멜빌	소설	10000	태양
14	바람과 함께 사라지다	마가렛 미첼	소설	12000	태양
15	실낙원	밀턴	소설	9000	강산
16	토지	박경리	소설	13000	문화

2 - 수필

id	제목	저자	카테고리	가격	출판사

3 - 경제

id	제목	저자	카테고리	가격	출판사
17	경제학 개론	홍길동	경제	10000	문화
18	미시경제학	임꺽정	경제	9000	태양
19	거시경제학	은달	경제	9000	대지
20	자본시장론	평강	경제	9500	대지
29	환율의 미래	전우치	경제	11000	대지
30	ETF 투자	홍길동	경제	9000	대지
31	에너지 혁명	은달	경제	8000	바다

4 - 과학

id	제목	저자	카테고리	가격	출판사
21	지질학개론	홍길동	과학	10000	지구
22	천문학개론	임꺽정	과학	1500	대지